

```
#List (Creation, Modification and Access):
#1. List Creation:
#a. Create a list named age_list with five integer elements. For eg., [24, 25, 26, 27, 28]
#b. Create a list named name_list with five string elements.
age_list=[24,25,26,27,28]
name_list=["Yash","Krish","Nani","Bunny","Deepu"]
print(age_list)
print(name_list)
```

```
[24, 25, 26, 27, 28]
['Yash', 'Krish', 'Nani', 'Bunny', 'Deepu']
```

```
#2. List Operations / Modifications:
#a. Append the string "Yazhini" to name_list.
name_list.append("Yazhini")
print(name_list)
#b. Insert the element 30 at index 2 in age_list.
age_list.insert(2,30)
print(age_list)
#c. Remove the string "Yazhini" from name_list.
name_list.remove("Yazhini")
print(name_list)
#d. Pop the last element from age_list.
age_list.pop()
print(age_list)
#e. Extend the age_list with additional ages [29, 30, 26].
age_list.extend([29,30,26])
print(age_list)
#f. Sort age_list in descending order.
age_list.sort(reverse=True)
print(age_list)
#g. Find Max age, Min age and sum of all ages from age_list.
print(max(age_list))
print(min(age_list))
print(sum(age_list))
```

```
['Yash', 'Krish', 'Nani', 'Bunny', 'Deepu', 'Yazhini']
[24, 25, 30, 26, 27, 28]
['Yash', 'Krish', 'Nani', 'Bunny', 'Deepu']
[24, 25, 30, 26, 27]
[24, 25, 30, 26, 27, 29, 30, 26]
[30, 30, 29, 27, 26, 26, 25, 24]
30
24
217
```

```
#3. Accessing List Elements:
#a. Print the first element of name_list.
#b. Print the last element of name_list.
#c. Print the elements from index 2 to index 4 in name_list.
#d. Print the elements of name_list in reverse order.
print("First name:",name_list[0])
print("Last name:",name_list[-1])
print(name_list[2:5])
print(name_list[::-1])
```

```
First name: Yash
Last name: Deepu
```

```
['Nani', 'Bunny', 'Deepu']
['Deepu', 'Bunny', 'Nani', 'Krish', 'Yash']
```

```
#Dictionary (Creation, Modification and Access):
#a. Create a dictionary named student_marks that maps the names of five students to their marks (u
#b. Access and print the mark of a specific student, of your choice.
#c. Add a new student "Janani" with a mark of 80 to the student_marks dictionary.
#d. Update the mark of any one older student to 82.
#e. Use the keys(), values(), and items() methods to print all keys, values, and key-value pairs i

student_marks={"Yash":90,"Krish":80,"Nani":70,"Bunny":60,"Deepu":50}
print(student_marks)
a=input("enter student name")
print(student_marks[a])
student_marks["Janani"]=80
print(student_marks)
student_marks["Yash"]=82
print(student_marks)
print(student_marks.keys())
print(student_marks.values())
print(student_marks.items())
```

```
{'Yash': 90, 'Krish': 80, 'Nani': 70, 'Bunny': 60, 'Deepu': 50}
enter student nameKrish
80
{'Yash': 90, 'Krish': 80, 'Nani': 70, 'Bunny': 60, 'Deepu': 50, 'Janani': 80}
{'Yash': 82, 'Krish': 80, 'Nani': 70, 'Bunny': 60, 'Deepu': 50, 'Janani': 80}
dict_keys(['Yash', 'Krish', 'Nani', 'Bunny', 'Deepu', 'Janani'])
dict_values([82, 80, 70, 60, 50, 80])
dict_items([('Yash', 82), ('Krish', 80), ('Nani', 70), ('Bunny', 60), ('Deepu', 50), ('Janani', 80)])
```

```
#Sets (Operations):
#a. Create a set called my_set with following values: ['a','e','i','o','u','a','a','i'] Analyse t
#b. Attempt to change the value of my_set[4] = 's'. If code throws an error, provide an explanati
#c. Create two sets:
#set1 with values: {1, 3, 5, 7, 9}
#set2 with values: {2, 3, 5, 8, 10}
#d. Compute and print the union and intersection of set1 and set2.
```

```
my_set={'a','e','i','o','u','a','a','i'}
print(my_set)
```

```
{'i', 'a', 'u', 'o', 'e'}
```

```
my_set={'a','e','i','o','u','a','a','i'}
my_set[4]='s'
print(my_set)
```

```
#TypeError: 'set' object does not support item assignment
```

```
#Sets do not support indexing
```

```
#For example, this works with a list:
```

```
my_list=['a', 'e', 'i', 'o', 'u']
my_list[4]= 's'
print(my_list)
```

```
['a', 'e', 'i', 'o', 's']
```

```
set1={1,3,5,7,9}
set2={2,3,5,8,10}
print(set1)
print(set2)
```

```
{1, 3, 5, 7, 9}
{2, 3, 5, 8, 10}
```

```
#d. Compute and print the union and intersection of set1 and set2.
set1={1,3,5,7,9}
set2={2,3,5,8,10}
print(set1.union(set2))
print(set1.intersection(set2))
```

```
{1, 2, 3, 5, 7, 8, 9, 10}
{3, 5}
```

```
#Operators & Conditional Statements : (IF, ELIF, ELSE)
#Performance Category Program:
#1. Prompt user for Input. Score range should be from 0 to 10 (both inclusive).
#2. Find the performance category based on the input score using following criteria:
#a. Above Average: Score greater than 7
#b. Average: Score between 4 and 7(both inclusive)
#c. Below Average: Score lesser than 4
#3. Output: Print the Performance category
#4. Additional Step: You can give a prompt of your choice to each category.
#For eg: If score below average "Need to Improve your performance, consistent practice will lead
```

```
score = float(input("Enter your score (0-10): "))

if score > 7:
    print("Performance Category: Above Average")
    print("Excellent performance! Keep up the good work.")

elif score >= 4:
    print("Performance Category: Average")
    print("Good effort! Keep practicing to improve further.")

else:
    print("Performance Category: Below Average")
    print("Need to improve your performance. Consistent practice will lead to better results.")
```

```
Enter your score (0-10): 7
Performance Category: Average
Good effort! Keep practicing to improve further.
```

